BH/1 — HTTP, in binary

Protocol specification, version 1 · Sanaa Ara

BH/1 carries HTTP's semantics (methods, paths, status codes, header fields) as length-prefixed binary frames over one TCP connection. This document is the whole protocol. `bserve` and `bcurl` are one implementation of it; an implementation built from this page alone must interoperate with them.

Conventions. Multi-byte integers are unsigned, big-endian. An *octet string* is raw bytes whose length travels in front of it; nothing is terminated. MUST, SHOULD and MAY are used as in RFC 2119. A *receiver* is whichever side is reading.

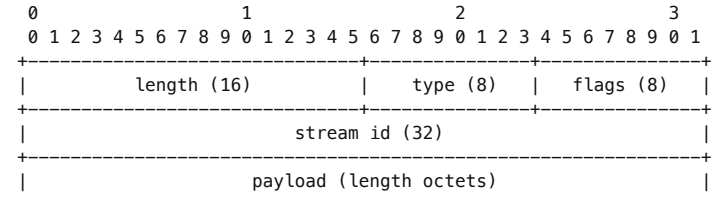
1. Connection

The client opens one TCP connection and writes the 4-byte **preface** `42 48 2f 31` (ASCII `BH/1`). Everything after it, in both directions, is a sequence of frames. The preface states the protocol version once per connection rather than once per frame. A server whose first four bytes are anything else closes the connection, which is also what happens to an HTTP/1.1 client that connects by mistake (`GET` is not `BH/1`).

The connection stays open across requests. Either side MAY close it when no request is in progress. A server SHOULD close a connection that has been idle for a while (`bserve`: 60 s); a client MUST treat that as normal.

2. Frames

Every frame is an 8-byte header followed by `length` bytes of payload.



Field	Bits	Meaning
length	16	Payload size, 0–65535. Every value the field can hold is legal, so a receiver's buffer is bounded by construction and nothing is negotiated.
type	8	`0x00` DATA, `0x01` HEADERS. `0x02`–`0xff` are undefined in BH/1.
flags	8	Bit 0 (`0x01`) is END_STREAM. Other bits are undefined; receivers MUST ignore them.
stream id	32	The request this frame belongs to (§4). `0` is the connection itself.

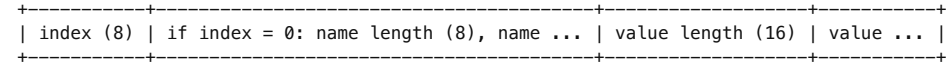
Unknown frame types. A receiver that meets a type it does not know MUST read and discard exactly `length` payload bytes, change no state, and reply with nothing. This is how a later version adds frames (settings, ping, go-away, push) without a new header layout: with the length at a fixed place in a fixed-size header, any frame can be stepped over whether or not its meaning is known. The header layout is therefore the one thing a version 2 may not change.

DATA (`0x00`). The payload is body bytes. END_STREAM marks the last frame of a message. A DATA frame of length 0 is legal; it is a way to send END_STREAM on its own.

HEADERS (`0x01`). The payload is one header block (§3), which MUST fit in the frame. 65535 bytes is thus the limit on a header block; HTTP servers default to 8 KiB, so no continuation mechanism is needed. END_STREAM means the message has no body. A message has exactly one HEADERS frame.

3. Header blocks

A header block is a sequence of fields, parsed until the payload ends; there is no field count, because the frame length already says where the block stops. Each field is a name followed by a value:



- `index` 1–10 names a static-table entry (below); the name itself is not sent.
- `index` 0 means a literal name follows: a one-byte length (1–255), then the name.
- A two-byte value length and the value always follow, whichever way the name was given.
- `index` 11–255 is undefined and MUST be treated as malformed. Such a field *could* be skipped, since its value is length-prefixed, but a receiver that drops a field whose name it cannot see is acting on a message it has not understood. HPACK makes the same call: an index outside the table is a decoding error. The table is part of the version the preface names.

1	2	3	4	5	6	7	8	9	10
`:method`	`:path`	`:status`	`host`	`user-agent`	`accept`	`content-type`	`content-length`	`date`	`server`

These are the names on every request (1, 2, 4, 5, 6) and every response (3, 7, 8, 9, 10) this implementation sends, and the names nearly every HTTP exchange carries. Anything else (`last-modified`, `cache-control`, …) is sent literal. On the exchange in HEXDUMP.md the two header blocks total 185 bytes; with every name literal they would be 270. A bigger table would save little more, which is also why there is no dynamic table.

Rules for fields:

- Literal names are lower-case tokens (RFC 9110 `tchar`, upper case excluded) and MUST NOT begin with `:`. The only pseudo-headers are the three in the table.
- Pseudo-headers come first, each at most once. A request carries `:method` and `:path` and no `:status`; a response carries `:status` and neither of the others.

- `:method` is a token (`GET`, `HEAD`). `:path` begins with `/`, is printable ASCII (`0x21`–`0x7e`), percent-encoded as in HTTP, and may end in `?query`. `:status` is exactly three ASCII digits.
- `content-length`, if present, is a decimal number and MUST equal the number of body bytes. It is advisory: the body is delimited by framing, so a mismatch is an error, not an instruction.
- Every other value is an opaque octet string of up to 65535 bytes. Regular names may repeat.

4. Requests and responses

Streams. A request and its response form a *stream*, named by the stream id carried in all their frames. The client numbers streams with odd ids, strictly increasing from 1 (1, 3, 5, …); the server answers on the same id. Even ids are reserved for streams a server might open in a later version; 0 is the connection.

Sequencing. All frames of one request are sent before any frame of the next. A client MAY send its next request before the previous response has arrived; the server answers streams in increasing id order.

Request. One HEADERS frame. If it lacks END_STREAM, DATA frames follow on the same stream until one carries END_STREAM, and the server MUST read them all before answering. `bserve` implements GET and HEAD and answers any other method with 405.

Response. One HEADERS frame, then DATA frames until END_STREAM; or a HEADERS frame carrying END_STREAM when there is no body (HEAD, or an empty file). `bserve` sends `content-type`, `content-length`, `date` and `server` on every response, plus `last-modified` on a 200. Status codes keep their HTTP meanings; `bserve` uses 200, 400, 403, 404 and 405. Error responses to GET carry a `text/plain` body holding the status line.

Path mapping. The server strips the query, percent-decodes, and resolves the path under its root. A path that leaves the root through `..` or a symbolic link is answered 404, as is anything that is not a regular file. A directory maps to the `index.html` inside it.

Closed streams. A stream is closed once the server has sent its END_STREAM. A receiver MUST ignore DATA frames on a closed stream: they are the tail of a message already dealt with, which is what lets a server answer 400 to a request whose body is still arriving without losing its place.

5. Errors

There are two kinds, as in HTTP/2.

A **malformed request** is a HEADERS frame whose block breaks §3, or a stream whose first frame is DATA. The server answers that stream with a 400, END_STREAM set, ignores any further DATA on it, and keeps the connection open. Because every frame is self-delimiting, a bad payload never costs the server its position in the byte stream.

A **connection error** is anything that cannot be answered on a stream: a wrong preface; a HEADERS or DATA frame on stream 0 or on an even id; a HEADERS frame whose id is not above every earlier id; or a frame that interrupts an incomplete request (a HEADERS frame, or DATA on a newer stream). The server closes the connection. BH/1 has no go-away frame; a version 2 can add one on stream 0, and a BH/1 receiver will skip it.

A client that receives a malformed response — an undecodable block, DATA before HEADERS, a known frame on a stream it did not open, a `content-length` that disagrees with the body, or end of file inside a frame — MUST abandon the connection. `bcurl` reports the reason and exits 1.

6. Why these widths

HTTP/2's frame header is 24 (length) / 8 (type) / 8 (flags) / 1 + 31 (stream id), nine bytes. The 24-bit length exists because HTTP/2 multiplexes: frames default to 16 KiB so that interleaved streams share the connection fairly, yet may grow to 16 MiB for bulk transfer, and that range must be negotiated through SETTINGS, since a receiver has to know how much to buffer before it reads. Type and flags get a byte each because a byte is the smallest unit worth addressing and 256 values is ample. The 31-bit stream id is sized for many concurrent streams on a long-lived connection; the reserved top bit keeps ids positive in languages with signed 32-bit integers, an inheritance from SPDY. The 9-byte total is an acknowledged wart: every payload starts unaligned. FastCGI, the other framed protocol in this course, uses eight bytes — version, type, 16-bit request id, 16-bit length, padding, reserved — and ends a stream with an empty record.

BH/1 uses 16 / 8 / 8 / 32, eight bytes:

- **16-bit length.** One connection, one request at a time: there is no interleaving to keep fair, so no reason to make frames small, and no reason to exceed 64 KiB, where the header is 0.012% of the frame and a 1 GiB file costs 128 KiB in headers. Letting the maximum be whatever the field can hold removes a class of errors (frame too large) and the settings exchange HTTP/2 needs to agree on a limit.
- **8-bit type, 8-bit flags.** As in HTTP/2, for the same reasons. Unknown types are skipped, unknown flags ignored: the two extension points.
- **32-bit stream id.** It costs four bytes a frame: 6% of the 63-byte request in HEXDUMP.md, 0.006% of a full DATA frame. It buys three things. Each response names its request, so a client can detect a desynchronised connection instead of silently reading someone else's answer, the failure that made HTTP/1.1 pipelining undeployable. Frames of a rejected request can be dropped by id instead of by guesswork. And version 2 can multiplex without touching the header. 32 bits rather than 31 because the header then packs into exactly eight bytes, and nothing here needs the reserved bit.
- **No version field, no padding, no end-of-stream record.** The version is a property of the connection, so it is sent once in the preface rather than in every frame, where FastCGI spends a byte per record on it. End of stream is a flag on the last frame, as in HTTP/2, rather than FastCGI's extra empty record: one bit instead of eight bytes and another pass through the parser.

References. RFC 9113 (HTTP/2); RFC 7541 (HPACK, §2.3 and §6.2 for the static table and literal representations); RFC 9110 (HTTP semantics); the FastCGI specification (1996); RFC 9292 (Binary HTTP, a serialisation of messages rather than a connection protocol, and so not what BH/1 is).